

Ecommerce Shop API

Data Entry Guide

Every endpoint explained, the exact order to enter data,
with many complete examples.

API version 1.0.0 | 2026-09-17

Base URL (local): <http://127.0.0.1:8000>

Contents

1. Basics: URL, login, envelopes	3
2. The data-entry flow (what to enter first)	4
3. Endpoint reference	5
3.1 Admin auth	5
3.2 Currencies - step 1 of the flow	6
3.3 Categories - step 2 of the flow	7
3.4 Product models - step 3 of the flow	8
3.5 Attributes - step 4 of the flow	8
3.6 Attribute values - step 5 of the flow	10
3.7 Products - step 6 of the flow	11
3.8 Image slots (how pictures work)	13
3.9 Standalone values, variants, options, images	13
3.10 Public endpoints (the shop website)	14
4. Complete examples (copy and adapt)	14
Example A - a slipper shop, start to finish	15
Example B - boots via bulk values + upload-then-reference	17
Example C - the smallest possible product	19
Example D - growing a product later (PATCH)	20
Example E - all three image flows side by side	20
Example F - error gallery (what went wrong?)	21
5. One-page cheat sheet	22

1. Basics: URL, login, envelopes

1.1 Base URL and versions

Every endpoint in this guide lives under **/api/v1/**. On your own machine the full address looks like this:

```
http://127.0.0.1:8000/api/v1/admin/products/  
http://127.0.0.1:8000/api/v1/products/
```

There are two families: **Public** (`/api/v1/...`) - read-only, no login, used by the shop website; and **Admin** (`/api/v1/admin/...`) - read and write, login required, used for data entry. This guide focuses on Admin, because that is where you type data in.

1.2 Login and the Bearer token (do this first)

Step 0, before any data entry: log in as a staff user. Send your email and password:

POST /api/v1/admin/auth/login/ (request)

```
{  
  "email": "admin@example.com",  
  "password": "change-me"  
}
```

Response 200

```
{  
  "success": true,  
  "message": "Login successful",  
  "data": {  
    "access": "eyJhbGciOi... (long token, valid 60 minutes)",  
    "refresh": "eyJhbGciOi... (valid 7 days)",  
    "user": {  
      "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",  
      "email": "admin@example.com",  
      "first_name": "",  
      "last_name": "",  
      "is_staff": true,  
      "is_active": true,  
      "date_joined": "2026-09-17T10:00:00Z"  
    }  
  }  
}
```

Copy the **access** token (not the refresh token). Every admin request must carry it in the header:

Authorization: Bearer <paste the access token here>

When the access token expires after 60 minutes, get a new one without retyping your password:

POST /api/v1/admin/auth/refresh/

```
{  
  "refresh": "eyJhbGciOi..."  
}
```

Only staff users can log in here. A normal (non-staff) account gets `Admin access required.`

1.3 The response envelope (always the same shape)

Every success looks like this - your payload is always inside **data**:

```
{
  "success": true,
  "message": "Created successfully.",
  "data": {
    "id": "..."
  }
}
```

Every error looks like this - field problems are listed inside **errors**:

```
{
  "success": false,
  "message": "Validation failed.",
  "errors": {
    "gender": [
      "Select a valid choice. UNISEX is not one of the available choices."
    ]
  }
}
```

- List endpoints put pagination inside `data`: `{count, next, previous, results}`.
- Successful **DELETE** returns **200** (not 204) with `{ "success": true, "message": "Deleted successfully.", "data": {} }`.
- List endpoints support `?page=` and `?page\_size=`, plus `?search=` and `?ordering=` where documented.

1.4 Three simplifications that make typing easy

- **Human keys instead of UUIDs.** Anywhere a reference is required you may pass a slug, name, code or SKU instead of a UUID: category/model/product slugs (`"slippers"`), attribute names (`"Color"`), value names resolved within the attribute (`"Grey"`), currency codes (`"NPR"`), variant SKUs (`"CELSI-GREY-40"`). Matching is exact-first with a single-match case-insensitive fallback, so `"npr"` and `"Slippers"` also work. UUIDs keep working everywhere.
- **Images in one JSON step.** Image slots accept `{ "file": "data:image/png;base64,..." }` (png/jpg/webp/gif, max 5 MB by default), so a product with pictures needs a single request - no pre-upload, no multipart. The classic two-step flow (upload, then reference the URL) still works.
- **Attribute + values in one call.** `POST /admin/attributes/` accepts `{ "value": ["Grey", "Blue"] }` and creates the names together with the attribute. Updates add missing names without complaining about existing ones.

2. The data-entry flow (what to enter first)

Enter data in this order. Each step feeds the next one - that is why the order matters:



2.1 Why this order (the dependency map)

- **1. Currency first** - every variant price needs one (`NPR`). Nothing depends on anything before it.
- **2. Category** - products must sit on one (`Slippers`). Needs nothing except optionally an image.

- **3. Product model** - the product line (`Celsi`). Its first word seeds auto-SKUs (`CELSI-...`). Needs nothing.
- **4. Attribute** - facets like `Color` or `Size`. Set **requires_image** now if every active value must carry a picture (typical for Color). Needs nothing.
- **5. Attribute values** - concrete options under an attribute (`Grey`, `Blue` under Color; `40`, `41` under Size). Needs its attribute (or create both in one call, section 4.5).
- **6. Product (last)** - ties everything together: pick category + model, attach values (with images where required), add variants priced in a currency. Needs steps 1-5.

Note: If a dropdown or reference looks empty while entering a product, you skipped an earlier step. Go back, add the missing piece, then continue - the product form does not create currencies, categories, models or values for you.

2.2 Rules a data-entry person must know

- **Names are unique:** category, model, attribute, and currency code. Value names are unique **per attribute** (Size can have `40` and another attribute can also have `40`).
- **Slugs and SKUs auto-generate** when left blank, and stay stable afterwards. You only type them to override.
- **requires_image** (e.g. on Color) forces every **active** value of that attribute to carry a feature image. Inactive values are exempt; variants can only use active values.
- **Variant rules:** at least 1 option; only one value per attribute (Grey+Blue together is rejected); no two variants of a product may repeat the same combination; every option must belong to the same product.
- **Nothing is ever implicitly deleted.** PATCH/PUT lists upsert by `id`; omitted values/variants are left untouched. Delete explicitly with DELETE.
- **Some deletes are blocked while in use** (category/model/currency, values inside products). The API answers 400 with `This record is in use...` - deactivate instead, or delete the referencing records first.
- **Prices** are decimals (`"5995.00"`), 0 or greater, always paired with a currency.
- **additional_images** always replaces the whole gallery on update: send the complete list you want to keep.

3. Endpoint reference

Every endpoint below shows its method, path, who may call it, what it is for, and its fields. Admin endpoints need `Authorization: Bearer <access>`. Public endpoints need nothing.

3.1 Admin auth (staff users only)

POST `/api/v1/admin/auth/login/`

Auth: none (public door to the admin API).

Log in with email + password. Returns the **access** token (60 min), **refresh** token (7 days) and your profile. Non-staff users are rejected.

email (string, required) Staff account email.

password (string, required) Account password.

POST `/api/v1/admin/auth/refresh/`

Auth: none (needs a valid refresh token).

Trade a refresh token for a fresh access token when the old one expires.

POST `/api/v1/admin/auth/logout/`

Auth: logged-in user.

Blacklist a refresh token (true logout). Request: `{"refresh": "..."}\``. Response data is `{}`.

GET `/api/v1/admin/auth/me/`

Auth: staff user.

Return the current admin profile (id, email, names, is_staff, is_active, date_joined).

POST `/api/v1/admin/auth/change-password/`

Auth: staff user.

Change your own password. Rejects a wrong old password, a new password equal to the old one, or a weak password.

old_password (string, required) Current password.

new_password (string, required) New password (validated for strength).

new_password_confirm (string, optional) If sent, must exactly match new_password.

3.2 Currencies - step 1 of the flow

Money that variant prices are quoted in. Create at least one (e.g. NPR) before any variant. Codes are uppercased automatically.

GET `/api/v1/admin/currencies/`

Auth: staff user.

List currencies. Search `?search=` (code, name), filter `?is_active=`, order by code/name/created_at.

POST `/api/v1/admin/currencies/`

Auth: staff user.

Create a currency.

code (string, required) E.g. ``NPR``. Auto-uppercased, unique.

name (string, required) E.g. ``Nepalese Rupee``.

symbol (string, required) E.g. ``Rs.``

is_active (boolean, optional) Default true. Inactive currencies cannot price new variants.

Request

```
{
  "code": "NPR",
  "name": "Nepalese Rupee",
  "symbol": "Rs."
}
```

GET `/api/v1/admin/currencies/{id}/`

Auth: staff user.

Retrieve one currency by UUID.

PUT `/api/v1/admin/currencies/{id}/`

Auth: staff user.

Replace a currency (send all fields).

PATCH **/api/v1/admin/currencies/{id}/**

Auth: staff user.

Partially update (e.g. `{"is_active": false}` to retire a currency).**DELETE** **/api/v1/admin/currencies/{id}/**

Auth: staff user.

Delete a currency. **Blocked (400)** while any variant is priced in it.

3.3 Categories - step 2 of the flow

Shelves that products live on (`Slippers`, `Boots`). Each category carries at most one image plus title/caption/alt metadata.

GET **/api/v1/admin/categories/**

Auth: staff user.

List categories (each row includes `product\_count`). Search name/description/slug, filter `?is\_active=`.

POST **/api/v1/admin/categories/**

Auth: staff user.

Create a category, with or without an image.

name (string, required) Unique. E.g. `Slippers`.**slug** (string, optional) Blank = auto from name (`slippers`).**description** (string, optional) Free text.**is_active** (boolean, optional) Default true. Inactive categories disappear from the public shop.**image** (image slot, optional) null/omitted = blank. See image shapes in section 3.8.**Request - plain category**

```
{
  "name": "Boots",
  "description": "Sturdy boots."
}
```

Request - with an inline picture (single step)

```
{
  "name": "Sandals",
  "image": {
    "file": "...",
    "title": "Sandals",
    "alt": "Sandals"
  }
}
```

GET **/api/v1/admin/categories/{id}/**

Auth: staff user.

Retrieve one category.

PUT **/api/v1/admin/categories/{id}/**

Auth: staff user.

Replace a category (all fields).

PATCH **/api/v1/admin/categories/{id}/**

Auth: staff user.

Partially update. Omit `image` to keep it, send `null` to clear it.

DELETE **/api/v1/admin/categories/{id}/**

Auth: staff user.

Delete a category. **Blocked (400)** while any product sits on it.

3.4 Product models - step 3 of the flow

Product lines such as `Celsi`. The first word of the model name seeds auto-generated SKUs (`CELSI-GREY-40`).

GET **/api/v1/admin/product-models/**

Auth: staff user.

List models (each row includes `product\_count`). Search name/description/slug, filter `?is\_active=`.

POST **/api/v1/admin/product-models/**

Auth: staff user.

Create a model.

name (string, required) Unique. E.g. `Celsi`.**slug** (string, optional) Blank = auto from name.**description** (string, optional) Free text.**is_active** (boolean, optional) Default true.**Request**

```
{
  "name": "Celsi",
  "description": "Wool felt slipper line."
}
```

GET **/api/v1/admin/product-models/{id}/**

Auth: staff user.

Retrieve one model.

PUT **/api/v1/admin/product-models/{id}/**

Auth: staff user.

Replace a model.

PATCH **/api/v1/admin/product-models/{id}/**

Auth: staff user.

Partially update a model.

DELETE **/api/v1/admin/product-models/{id}/**

Auth: staff user.

Delete a model. **Blocked (400)** while any product uses it.

3.5 Attributes - step 4 of the flow

Facets like `Color` or `Size`. Flip **requires_image** on when every active value must carry a feature image (typical for Color - no code change needed for new image-requiring attributes). You can create the attribute **and its values in one call** with `"value": [...]`.

GET **/api/v1/admin/attributes/**

Auth: staff user.

List attributes (rows include `values\_count` and the `values` name list). Search name, filter `?is\_active=` and `?requires\_image=`.

POST **/api/v1/admin/attributes/**

Auth: staff user.

Create an attribute, optionally with nested value names (all-or-nothing).

name (string, required) Unique. E.g. `Color`.

requires_image (boolean, optional) Default false. When true, active values need a feature image.

is_active (boolean, optional) Default true.

value (string[], optional, write-only) Up to 200 names created with the attribute, e.g. `["Grey","Blue"]`. Duplicates or blanks fail the whole request.

Request - attribute + values in one call

```
{
  "name": "Color",
  "requires_image": true,
  "value": [
    "Grey",
    "Blue"
  ]
}
```

Response 201 (data)

```
{
  "id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
  "name": "Color",
  "requires_image": true,
  "is_active": true,
  "values_count": 2,
  "values": [
    "Blue",
    "Grey"
  ],
  "created_at": "2026-09-17T10:00:00Z",
  "updated_at": "2026-09-17T10:00:00Z"
}
```

GET **/api/v1/admin/attributes/{id}/**

Auth: staff user.

Retrieve one attribute with its values list.

PUT **/api/v1/admin/attributes/{id}/**

Auth: staff user.

Replace an attribute; nested `value` names are ensured to exist.

PATCH **/api/v1/admin/attributes/{id}/**

Auth: staff user.

Partially update. Nested `value` names are **added if missing** and existing ones are left untouched (never an error, never deleted).

DELETE `/api/v1/admin/attributes/{id}/`

Auth: staff user.

Delete an attribute. Its global values go with it (CASCADE), but the delete is **blocked (400)** while any product value references it.

3.6 Attribute values - step 5 of the flow

Concrete options under an attribute: `Grey`/`Blue` under Color, `40`/`41` under Size. Unique per attribute. Create them one by one or in bulk.

GET `/api/v1/admin/attribute-values/`

Auth: staff user.

List values. Filter `?attribute=<uuid>` and `?is\_active=`, search value or attribute name.

POST `/api/v1/admin/attribute-values/`

Auth: staff user.

Create ONE value (`{attribute, name}`) or MANY at once (`{attribute, value: [...]}` - same as the bulk endpoint). Send `name` or `value`, never both.

attribute (uuid or name, required) E.g. `Color` or the attribute UUID.

name (string, single shape) The value name, e.g. `Grey`.

value (string[], bulk shape) 1-200 names, all-or-nothing. Duplicates or existing names fail everything.

is_active (boolean, optional) Single shape only. Default true.

Request - single

```
{
  "attribute": "Color",
  "name": "Red"
}
```

Request - bulk (same URL)

```
{
  "attribute": "Color",
  "value": [
    "Red",
    "Black",
    "Blue"
  ]
}
```

POST `/api/v1/admin/attribute-values/bulk/`

Auth: staff user.

Bulk-only alias: `{attribute, value: [...]}` creates many values atomically and returns the created rows.

GET `/api/v1/admin/attribute-values/{id}/`

Auth: staff user.

Retrieve one value.

PUT `/api/v1/admin/attribute-values/{id}/`

Auth: staff user.

Replace one value.

PATCH `/api/v1/admin/attribute-values/{id}/`

Auth: staff user.

Partially update one value (rename, move attribute, toggle active).

DELETE `/api/v1/admin/attribute-values/{id}/`

Auth: staff user.

Delete a value. **Blocked (400)** while any product uses it.

3.7 Products - step 6 of the flow

The final step that ties everything together. One POST can carry the product, its attribute values (with images) and its priced variants - the whole write is atomic: any nested failure rolls everything back.

GET `/api/v1/admin/products/`

Auth: staff user.

List products (compact rows with nested category/model and `variants\_count`). Search name/slug/description; filter category slug (`?category=slippers`), gender, featured, price range, active; order by name/created_at/featured.

POST `/api/v1/admin/products/`

Auth: staff user.

Create a product with nested values + variants. Temp `key`s let variants point at values created in the same request. Returns the full read shape (201).

name (string, required) E.g. `Celsi Wool Felt Slippers`.**model** (uuid or slug, required) E.g. `celsi`.**gender** (choice, required) One of MEN, WOMEN, UNISEX, KIDS, BABY.**category** (uuid or slug, required) E.g. `slippers`.**slug** (string, optional) Blank = auto from name.**description / general_information / materials_used** (string, optional) Free text blocks.**is_featured** (boolean, optional) Default false.**is_active** (boolean, optional) Default true.**key_features** (object[], optional) `[{"title": "Sole", "value": "Rubber"}]`.**attribute_values** (object[], optional) Nested values - see fields below.**variants** (object[], optional) Nested variants - see fields below.

Nested value fields (inside `attribute\_values`): `id` (omit = create, send = update that row), `key` (temp label variants point at, unique per request), `attribute` + `attribute\_value` (required on create; UUIDs or names like `Color`/`Grey`), `feature\_image` (image slot; **mandatory** when the attribute requires images and the row is active), `additional\_images` (gallery list), `is\_active` (default true).

Nested variant fields (inside `variants`): `id` (omit = create), `sku` (blank = auto like `CELSI-GREY-40`), `price` (required on create, e.g. `5995.00`), `currency` (required on create, UUID or code like `NPR`), `is\_special\_edition` / `is\_active`, `options` (required on create, at least one: a value UUID, a temp key `grey`, or `{id:...}`/`key:...` objects).

Request - full product in one call

```
{
  "name": "Celsi Wool Felt Slippers",
  "model": "celsi",
  "gender": "UNISEX",
  "description": "Warm wool felt slippers.",
  "category": "slippers",
```

```

"is_featured": true,
"key_features": [
  {
    "title": "Sole",
    "value": "Rubber"
  }
],
"attribute_values": [
  {
    "key": "grey",
    "attribute": "Color",
    "attribute_value": "Grey",
    "feature_image": {
      "file": "...",
      "title": "Grey",
      "alt": "Grey slippers"
    }
  },
  {
    "key": "40",
    "attribute": "Size",
    "attribute_value": "40"
  }
],
"variants": [
  {
    "price": "5995.00",
    "currency": "NPR",
    "options": [
      {
        "key": "grey"
      },
      {
        "key": "40"
      }
    ]
  }
]
}

```

GET `/api/v1/admin/products/{id}/`

Auth: staff user.

Retrieve one product fully expanded (values with images, variants with priced options, category/model details, creator).

PUT `/api/v1/admin/products/{id}/`

Auth: staff user.

Update a product. Nested lists **upsert by id**; items without `id` are created; omitted relations are never deleted.

PATCH `/api/v1/admin/products/{id}/`

Auth: staff user.

Partially update - e.g. add one variant, change a price, or attach a new value - everything else untouched.

DELETE**/api/v1/admin/products/{id}/**

Auth: staff user.

Delete a product with all its values, variants and images (stored assets are cleaned up too).

3.8 Image slots (how pictures work)

``image`` (category), ``feature_image`` and ``additional_images`` (product values) all follow one contract. **Reads** return delivery URLs plus metadata; **writes** are flexible:

- ``null``, ```` or `{}`` clears a single image (omit the field to keep it).
- A plain URL string sets just the URL: ``image`: "https://..."``.
- An object references an existing asset: `{ "url"... }`, `{ "name"... }` or `{ "public_id"... }`, plus optional ``title``/``caption``/``alt``. Paste back what a previous read returned.
- Multipart uploads: send the binary file itself as the field (``image``, ``feature_image``, repeat ``additional_images`` for several).
- Inside JSON, upload inline: `{ "file": "data:image/png;base64,..." }` - no pre-upload needed.
- Inside nested multipart payloads, point at a file part: `{ "file": "grey-img" }` plus a `grey-img`` file part.

Read shape (what GET returns)

```
{
  "url": "https://res.cloudinary.com/demo/image/upload/v1/shop/grey-main.jpg",
  "public_id": "shop/grey-main",
  "title": "Grey",
  "caption": "Grey wool felt slippers",
  "alt": "Grey slippers"
}
```

Note: `additional_images` replaces the whole gallery on update: always send the complete list you want to keep, in order.

3.9 Standalone values, variants, options, images

Everything in section 3.7 can also be managed piece by piece. Useful for small fixes without touching the whole product.

GET / POST**/api/v1/admin/product-attribute-values/**

Auth: staff user.

List values (filter ``?product=`, `?attribute=`, `?is_active=`, search product/attribute/value names) or attach ONE value to a product. Fields: `product` (uuid/slug, immutable), `attribute` (uuid/name), `attribute_value` (uuid/name), `feature_image`, `additional_images`, `is_active`. The attribute/value must match, and image-requiring attributes need a feature image.`

GET / PUT / PATCH / DELETE**/api/v1/admin/product-attribute-values/{id}/**

Auth: staff user.

Read, replace, partially update or detach one product value. Values are never moved between products. Delete is **blocked (400)** while any variant option points at it.

GET / POST**/api/v1/admin/variants/**

Auth: staff user.

List variants (filter ``?product=`, `?currency=NPR`, `?is_special_edition=`, `?is_active=`, search sku/product) or create ONE variant. Fields: `product`, `sku` (blank = auto), `price`, `currency`, flags, and `options` = list of product-value UUIDs (required on create, at least one, one per attribute, unique combination).`

GET / PUT / PATCH / DELETE**/api/v1/admin/variant-options/{id}/**

Auth: staff user.

Read, replace, partially update or delete one variant. Variants are never moved between products; omit `options` on update to keep them.

GET / POST**/api/v1/admin/variant-options/**

Auth: staff user.

List single options (filter `?variant=`, `?product=`) or add ONE option to a variant: `{"variant": "<uuid-or-SKU>", "product\_attribute\_value": "<uuid>"}`. Same combination rules apply.

GET / PUT / PATCH / DELETE**/api/v1/admin/variant-options/{id}/**

Auth: staff user.

Read, repoint (`{"product\_attribute\_value": "<new uuid>"`), or remove one option. Removing must not empty the variant nor duplicate a sibling combination.

POST**/api/v1/admin/images/upload/**

Auth: staff user.

Upload one image file (`multipart/form-data`, field `image`, optional `folder`). Returns `{public\_id, url, width, height, format}` - paste the reference into any image slot. Validated: png/jpg/webp/gif, max 5 MB default.

POST**/api/v1/admin/images/delete/**

Auth: staff user.

Delete one uploaded asset: `{ "public\_id": "..."}` returns `{ "deleted": true/false}`. Normally you never need this - replacing or deleting records cleans their images automatically.

3.10 Public endpoints (the shop website)

Read-only, no login, active records only. Included so data-entry staff can verify what shoppers will see after each change.

GET**/api/v1/categories/ and /api/v1/categories/{slug}/**

Auth: none.

List categories (slug, image, product_count) or one category by slug. Search name/description.

GET**/api/v1/product-models/ and /api/v1/product-models/{slug}/**

Auth: none.

List product models or one model by slug.

GET**/api/v1/attributes/ and /api/v1/attributes/{id}/**

Auth: none.

List attributes with their values, or one attribute.

GET**/api/v1/products/**

Auth: none.

Product cards: price_range, primary_image. Filters
 `?category=slippers&gender=UNISEX&is\_featured=true&min\_price=&max\_price=`, `?search=`,
 `?ordering=price|-price|-created\_at|name`, pagination.

GET**/api/v1/products/{slug}/**

Auth: none.

Full product page: grouped attribute values with images, every variant with price/currency/edition and option ids (the frontend matches the shopper's selection to find the SKU for WhatsApp checkout).

4. Complete examples (copy and adapt)

Five end-to-end stories plus an error gallery. All requests are JSON with `Authorization: Bearer <access>` unless stated otherwise.

Example A - a slipper shop, start to finish

A1. Currency. Without this, no price can be saved.

POST /api/v1/admin/currencies/

```
{
  "code": "NPR",
  "name": "Nepalese Rupee",
  "symbol": "Rs."
}
```

A2. Category with a picture in one step (inline base64 upload).

POST /api/v1/admin/categories/

```
{
  "name": "Slippers",
  "description": "Comfy everyday slippers.",
  "image": {
    "file": "...",
    "title": "Slippers",
    "alt": "Slippers"
  }
}
```

A3. Product model.

POST /api/v1/admin/product-models/

```
{
  "name": "Celsi",
  "description": "Wool felt slipper line."
}
```

A4. Attributes with their values - two calls total.

POST /api/v1/admin/attributes/ (x2)

```
{
  "name": "Color",
  "requires_image": true,
  "value": [
    "Grey",
    "Blue"
  ]
}
{
  "name": "Size",
  "value": [
    "40",
    "41"
  ]
}
```

A5. The product - category + model by slug, values by name, one inline picture, three priced variants. Notice the temp `key`s (`grey`, `blue`, `40`, `41`) that variants point at.

POST /api/v1/admin/products/

```
{
  "name": "Celsi Wool Felt Slippers",
  "model": "celsi",
  "gender": "UNISEX",
  "description": "Warm wool felt slippers.",
  "general_information": "Handmade in Nepal.",
  "materials_used": "Wool felt, rubber sole.",
  "category": "slippers",
  "is_featured": true,
  "key_features": [
    {
      "title": "Upper Material",
      "value": "Wool Felt"
    },
    {
      "title": "Sole",
      "value": "Rubber"
    }
  ],
  "attribute_values": [
    {
      "key": "grey",
      "attribute": "Color",
      "attribute_value": "Grey",
      "feature_image": {
        "file": "...",
        "title": "Grey",
        "alt": "Grey slippers"
      }
    },
    {
      "key": "blue",
      "attribute": "Color",
      "attribute_value": "Blue",
      "feature_image": {
        "file": "...",
        "title": "Blue"
      }
    },
    {
      "key": "40",
      "attribute": "Size",
      "attribute_value": "40"
    },
    {
      "key": "41",
      "attribute": "Size",
      "attribute_value": "41"
    }
  ]
}
```

```

],
"variants": [
  {
    "price": "5995.00",
    "currency": "NPR",
    "options": [
      {
        "key": "grey"
      },
      {
        "key": "40"
      }
    ]
  },
  {
    "price": "5995.00",
    "currency": "NPR",
    "options": [
      {
        "key": "grey"
      },
      {
        "key": "41"
      }
    ]
  },
  {
    "price": "6495.00",
    "currency": "NPR",
    "is_special_edition": true,
    "options": [
      {
        "key": "blue"
      },
      {
        "key": "40"
      }
    ]
  }
]
}

```

Response is **201** with the full product: auto slug `celsi-wool-felt-slippers`, auto SKUs `CELSI-GREY-40`, `CELSI-GREY-41`, `CELSI-BLUE-40`, and delivery URLs for every picture. Verify on the public side with `GET /api/v1/products/celsi-wool-felt-slippers`.

Example B - boots via bulk values + upload-then-reference

B1. Upload the photo first (multipart form): `POST /api/v1/admin/images/upload/` with file field `image` and `folder=products`. Response data:

```

{
  "public_id": "products/a1b2c3.png",
  "url": "https://res.cloudinary.com/demo/image/upload/v1/products/a1b2c3.png",
  "width": 800,
  "height": 600,
  "format": "png"
}

```

B2. Attribute without values, then 3 values in one bulk call:

POST /api/v1/admin/attributes/ then POST /api/v1/admin/attribute-values/bulk/

```
{
  "name": "Material"
}
{
  "attribute": "Material",
  "value": [
    "Leather",
    "Suede",
    "Canvas"
  ]
}
```

B3. Category referencing the uploaded picture (paste the URL back):

POST /api/v1/admin/categories/

```
{
  "name": "Boots",
  "image": {
    "url": "https://res.cloudinary.com/demo/image/upload/v1/products/a1b2c3.png",
    "title": "Boots",
    "alt": "Boots"
  }
}
```

B4. Product using UUIDs for a change (both styles are valid), reusing the same picture object on the value:

POST /api/v1/admin/products/ (excerpt)

```
{
  "name": "Trail Boots",
  "model": "3fa85f64-5717-4562-b3fc-2c963f66afa1",
  "gender": "MEN",
  "category": "boots",
  "attribute_values": [
    {
      "key": "leather",
      "attribute": "3fa85f64-5717-4562-b3fc-2c963f66afa3",
      "attribute_value": "3fa85f64-5717-4562-b3fc-2c963f66afa4",
      "feature_image": {
        "public_id": "products/a1b2c3",
        "title": "Boots"
      }
    }
  ],
  "variants": [
    {
      "sku": "TRAIL-LEATHER",
      "price": "8995.00",
      "currency": "NPR",
      "options": [
        "leather"
      ]
    }
  ]
}
```

Here the SKU is typed explicitly (`TRAIL-LEATHER`) and the option uses the bare key string `"leather"` - all three option styles (UUID, bare key, `{key}` object) are equivalent.

Example C - the smallest possible product

Only truly-required fields. Slugs, SKUs, flags and timestamps fill themselves in:

POST /api/v1/admin/products/

```
{
  "name": "Kids Sandals",
  "model": "sandalino",
  "gender": "KIDS",
  "category": "sandals",
  "attribute_values": [
    {
      "attribute": "Size",
      "attribute_value": "30"
    }
  ],
  "variants": [
    {
      "price": "1995.00",
      "currency": "NPR",
      "options": [
        {
          "key": "30"
        }
      ]
    }
  ]
}
```

Wait - the variant points at key `"30"` but the value above has no `key`! Correct version: either give the value `"key": "30"`, or (after creating) point at its UUID. Keys are only needed when variants must reference values from the same request:

Corrected (add the key)

```
{
  "name": "Kids Sandals",
  "model": "sandalino",
  "gender": "KIDS",
  "category": "sandals",
  "attribute_values": [
    {
      "key": "30",
      "attribute": "Size",
      "attribute_value": "30"
    }
  ],
  "variants": [
    {
      "price": "1995.00",
      "currency": "NPR",
      "options": [
        "30"
      ]
    }
  ]
}
```

```
]
}
```

Example D - growing a product later (PATCH)

Size `42` just arrived for the Celsi slippers. First ensure the global value exists (idempotent - safe to repeat):

PATCH /api/v1/admin/attributes/{size-id}/

```
{
  "value": [
    "42"
  ]
}
```

Then attach it to the product and add the new variant - one PATCH, everything else untouched (ids below are illustrative):

PATCH /api/v1/admin/products/{product-id}/

```
{
  "attribute_values": [
    {
      "key": "42",
      "attribute": "Size",
      "attribute_value": "42"
    }
  ],
  "variants": [
    {
      "price": "5995.00",
      "currency": "NPR",
      "options": [
        {
          "key": "grey"
        },
        {
          "key": "42"
        }
      ]
    }
  ]
}
```

To change an existing variant's price instead, send its `id`: `{"variants": [{"id": "<variant-uuid>", "price": "6295.00"}]}`. To rename a value's picture, PATCH the standalone value (section 3.9) with the new `feature\_image` object.

Example E - all three image flows side by side

Same result - a feature image on a product value - three ways:

1) Inline base64 (1 JSON request - simplest)

```
{
  "feature_image": {
    "file": "...",
    "title": "Grey"
  }
}
```

2) Upload then reference (2 requests)

```
{
  "feature_image": {
    "url": "https://res.cloudinary.com/demo/image/upload/v1/shop/grey-main.jpg",
    "title": "Grey"
  }
}
```

3) Multipart in one request (for scripts):

```
curl -X POST http://127.0.0.1:8000/api/v1/admin/products/ \
-H "Authorization: Bearer $ACCESS" \
-F name="Wool Slippers" -F model="celsi" -F gender="UNISEX" -F category="slippers" \
-F attribute_values='[{"key": "grey", "attribute": "Color", "attribute_value": "Grey",
"feature_image": {"file": "grey-img"}}]' \
-F variants='[{"price": "5995.00", "currency": "NPR", "options": [{"key": "grey"}]]' \
-F grey-img=@grey.png
```

Rule of thumb: humans and dashboards use (1); file pipelines that already speak multipart use (3); (2) is handy when the same picture is reused in many places.

Example F - error gallery (what went wrong?)

Real error shapes you may meet, and the fix:

Missing feature image (Color requires one)

```
{
  "success": false,
  "message": "Validation failed.",
  "errors": {
    "attribute_values": {
      "0": {
        "feature_image": [
          "A feature image is required for 'Color' values."
        ]
      }
    }
  }
}
```

Fix: add `feature\_image` to that value, or deactivate the value.

Value name from the wrong attribute

```
{
  "success": false,
  "message": "Validation failed.",
  "errors": {
    "attribute_value": [
      "No value '40' for attribute 'Color'. Available for 'Color': Blue, Grey."
    ]
  }
}
```

Fix: pick a name from the suggested list, or pass the right `attribute` alongside the name.

Duplicate variant combination

```
{
  "success": false,
  "message": "Validation failed.",
  "errors": {
    "variants": {
```

```

    "1": {
      "options": [
        "This exact combination already exists for another variant of this product."
      ]
    }
  }
}
}

```

Fix: each variant needs a different option set - check the existing variants first.

Delete blocked (record in use)

```

{
  "success": false,
  "message": "Cannot delete this record because it is referenced by other records.",
  "errors": {
    "non_field_errors": [
      "This record is in use and cannot be deleted. Deactivate it instead, or remove the referencing records first."
    ]
  }
}

```

Fix: follow the message - deactivate, or delete the referencing products/variants first.

Bad login

```

{
  "success": false,
  "message": "Authentication required.",
  "errors": {
    "detail": "No active account found with the given credentials"
  }
}

```

Fix: check email/password and that the account is staff + active.

5. One-page cheat sheet

Order of entry

- 0. Login -> `Authorization: Bearer <access>`.
- 1. Currency (`NPR`) -> 2. Category (`Slippers` + image?) -> 3. Model (`Celsi`).
- 4. Attribute (`Color`, requires_image?) + 5. values (`Grey`, `Blue`) - one call with `"value": [...]`.
- 6. Product: category + model, values with keys + images, variants with price + currency + options.
- Verify on public `GET /api/v1/products/{slug}/`.

References (UUID or human key)

- Category / model / product: slug - `"slippers"`, `"celsi"`.
- Attribute: name - `"Color"`. Value: name within the attribute - `"Grey"`.
- Currency: code - `"NPR"`. Variant: SKU - `"CELSI-GREY-40"`.

Images

- Simplest: `{"file": "data:image/png;base64,..."}` with title/alt.
- Reuse: paste `{"url": ...}` or `{"public\_id": ...}` from any earlier read.
- `null` clears a single image; omitting keeps it; `additional\_images` always replaces fully.

Variants

- Options: UUID, `"key"`, or `{"key"} / {"id"}` - at least one per variant.
- Blank SKU auto-generates (`CELSI-GREY-40``) and never changes afterwards.
- Rules: one value per attribute, unique combination per product, active values only.

Safety

- Product writes are atomic - a failure rolls back the entire request.
- Updates never delete omitted relations; DELETE is explicit and returns 200.
- In-use records refuse deletion with a 400 - deactivate or unlink first.